Teste Técnico

Desenvolvedor Full Stack Sênior · Grupo Elevate

1. Prazo e dedicação

Prazo para entrega: 4 dias corridos após o recebimento do teste.

Tempo estimado de dedicação: aproximadamente 8 a 12 horas.

Você não precisa utilizar todo o período disponível. O prazo de 4 dias existe para que o teste possa ser conciliado com outras atividades profissionais.

2. Objetivo

O objetivo deste teste é avaliar como você desenvolve uma aplicação Full Stack, considerando não apenas a implementação, mas também:

- arquitetura e organização de código;
- modelagem de banco de dados;
- desenvolvimento de APIs;
- integração através de webhooks;
- segurança e concorrência;
- testes;
- controle de versão;
- tomada de decisões técnicas;
- a forma como você conduz o desenvolvimento com apoio de IA.

O teste simula um problema próximo das aplicações reais de operação e tecnologia do Grupo Elevate. O foco principal é engenharia de software; a interface não precisa ser visualmente sofisticada.

> Não esperamos uma aplicação gigantesca nem um produto pronto para produção. Queremos avaliar sua capacidade técnica, organização, raciocínio e tomada de decisões.

3. Uso de IA

O uso de ferramentas de IA é **esperado** neste teste. Não avaliamos se você usou, e sim **como** usou: como dividiu o problema, que contexto forneceu, o que revisou, o que corrigiu e o que decidiu por conta própria.

Você é responsável por tudo o que entregar. Na apresentação técnica, você deverá explicar, demonstrar e alterar qualquer parte do código.

Uma solução gerada de uma só vez a partir deste enunciado, sem revisão, não atende ao que buscamos.

4. Cenário

O Grupo Elevate trabalha com diferentes produtos, páginas de vendas, funis de conversão e fontes de aquisição. Quando um visitante preenche um formulário em uma página de vendas, um lead é gerado. Esse lead precisa ser:

- recebido pelo sistema;
- validado;
- armazenado;
- associado ao produto e ao funil de origem;
- identificado através dos dados de tracking;
- distribuído para uma operação de atendimento (call center);
- acompanhado durante seu ciclo de vida.

O sistema deverá possuir uma API para recebimento dos leads e uma interface administrativa para acompanhamento. Todos os nomes de produtos utilizados neste teste são fictícios.

5. Aplicação

Desenvolva uma aplicação chamada **Elevate Lead Dispatcher**. Ela deverá possuir, no mínimo:

```
Frontend → Backend / API → Database
```

Você poderá adicionar outros componentes (fila, worker, cache etc.) caso considere necessário. Nenhum deles é obrigatório. Se utilizar um componente adicional, explique no README por que ele foi utilizado.

Não será atribuída pontuação adicional simplesmente por utilizar uma arquitetura mais complexa.

6. Recebimento de leads

Crie o endpoint **POST /api/leads**. Exemplo de payload:

```
{
  "name": "João Silva",
  "email": "joao@email.com",
  "phone": "+5511999999999",
  "product": "nutralean",
  "funnel": "vsl-01",
  "source": "facebook",
  "campaign": "campaign-123",
  "affiliate_id": "999",
  "utm": {
    "source": "facebook",
    "medium": "cpc",
    "campaign": "campaign-123",
    "content": "ad-01"
  }
}
```

A API deverá:

- validar os dados recebidos;
- armazenar o lead;
- gerar um identificador único;
- registrar data e hora de criação;
- associar o lead ao produto e ao funil;
- registrar as informações de origem;
- retornar uma resposta HTTP adequada.

Exemplo de resposta:

```
{
  "success": true,
  "lead_id": "8f3c7c2e-5b1a-4d0e-9a67-2f4c1b7e9d10"
}
```

Os campos e estruturas podem ser modificados caso exista uma justificativa técnica.

7. Validação

A API deverá validar os dados recebidos. Considere, por exemplo: nome obrigatório, e-mail válido, telefone válido, produto existente, funil válido, estrutura correta de UTM e tipos de dados esperados.

A aplicação deve retornar mensagens de erro e status HTTP adequados para requisições inválidas.

8. Duplicidade e idempotência

Considere que a mesma requisição pode ser enviada mais de uma vez. O sistema não deve criar registros duplicados de maneira indevida.

A estratégia é de sua escolha (idempotency key, identificador externo, combinação de campos, unique constraint ou outra solução justificável). Explique no README como sua aplicação identifica e trata requisições duplicadas.

9. Banco de dados

Modele o banco de dados de acordo com sua interpretação do problema. O sistema deve representar pelo menos os seguintes conceitos:

- Lead
- Product
- Funnel
- Affiliate
- Call Center
- Distribution
- Lead Status

Você poderá criar outras entidades. O banco deverá permitir a consulta de leads, o relacionamento com produto, funil e origem/afiliado, a distribuição e o histórico de status.

As alterações de estrutura do banco devem ser versionadas (migrations).

10. Índices

Identifique quais campos precisam de índices e explique no README quais foram indexados, por quê e quais consultas pretende otimizar. Não é necessário criar índices indiscriminadamente.

11. Escalabilidade do banco

Considere que o sistema inicialmente terá 100.000 leads, mas poderá chegar a 1.000.000 e a mais de 10.000.000.

Explique como sua modelagem se comportaria nesse cenário. Não é necessário implementar uma arquitetura para 10 milhões de registros; queremos avaliar sua capacidade de pensar sobre o crescimento da aplicação.

12. Lead Dispatcher

Implemente um mecanismo para distribuir leads entre diferentes call centers. Considere inicialmente:

- Call Center A: 50%
- Call Center B: 30%
- Call Center C: 20%

A cada novo lead, o sistema deverá determinar para qual operação ele será encaminhado e registrar essa distribuição. A estratégia é de sua escolha (weighted round robin, distribuição proporcional, fila, algoritmo próprio). Explique a escolha.

> Não é necessário integrar com um call center real nem enviar o lead para um sistema externo: registrar a distribuição no sistema é suficiente. Se optar por simular o envio, documente como fez.

13. Concorrência

Considere 100 leads chegando praticamente ao mesmo tempo. O sistema deverá continuar funcionando corretamente, e a distribuição não deve apresentar resultados incorretos devido a condições de corrida, como duas requisições que consultam o mesmo estado e depois o atualizam.

Explique como sua implementação evita esse tipo de problema. Você poderá utilizar transactions, locks no banco, operações atômicas, Redis, filas, workers ou outra solução justificável.

14. Webhook do call center

Crie o endpoint **POST /api/webhooks/callcenter**, pelo qual o call center envia atualizações sobre um lead. Exemplo:

```
{
  "lead_id": "8f3c7c2e-5b1a-4d0e-9a67-2f4c1b7e9d10",
  "external_id": "abc123",
  "status": "contacted",
  "timestamp": "2026-10-08T12:30:00Z"
}
```

Status possíveis: **new, contacted, no_answer, callback, qualified, sale, refund, invalid**.

O sistema deverá localizar o lead, atualizar seu status, registrar o evento e manter histórico suficiente para auditoria.

15. Webhook duplicado

O mesmo webhook pode ser recebido várias vezes. O sistema deverá tratar corretamente essa situação. Explique como você garante que o mesmo evento não seja processado incorretamente mais de uma vez.

16. Eventos fora de ordem

Um evento **contacted** pode chegar depois de um evento **sale**. Explique como sua aplicação lida com essa situação.

Não existe necessariamente uma única resposta correta. O importante é demonstrar que você percebe o problema de ordenação de eventos e de transição de estados.

17. Segurança do webhook

O sistema deverá possuir alguma estratégia para evitar que qualquer pessoa consiga enviar atualizações falsas para a API (HMAC, API key, assinatura, token ou outra solução justificável). Explique sua escolha.

18. Dashboard administrativo

Crie uma interface administrativa que apresente pelo menos:

- Total de leads
- Leads hoje
- Leads distribuídos
- Leads pendentes
- Vendas
- Taxa de conversão

Também deverá existir uma tabela de leads com, por exemplo: lead, produto, funil, origem, call center, status e data.

A interface não precisa ser sofisticada. Priorize funcionalidade e organização.

19. Filtros

A tabela deverá permitir filtrar os leads por período, produto, funil, origem, call center e status. Utilize paginação: não carregue todos os leads de uma vez no navegador.

20. Autenticação

Implemente autenticação com pelo menos dois perfis.

ADMIN

- visualiza todos os leads e as métricas;
- consulta a distribuição;
- administra as configurações relacionadas ao dispatcher.

OPERATOR

Cada OPERATOR está vinculado a um call center.

- visualiza apenas os leads distribuídos para o seu call center;

- atualiza o status desses leads;
- consulta as informações necessárias para o atendimento.

O OPERATOR não deve possuir acesso às funções administrativas nem aos leads de outros call centers.

21. Autorização

Não basta esconder funcionalidades no frontend: o backend deverá verificar as permissões do usuário. Um OPERATOR não deve conseguir acessar uma rota administrativa, ou um lead de outro call center, alterando a URL ou fazendo uma requisição manual para a API.

22. Segurança da aplicação

Considere pelo menos: autenticação, autorização, SQL Injection, XSS, CSRF (quando aplicável), brute force, rate limiting, exposição de dados sensíveis, validação de entrada e segurança dos webhooks.

Não é necessário implementar todas as proteções possíveis. O importante é demonstrar que você sabe identificar os riscos relevantes.

23. Testes

Implemente testes automatizados para as principais funcionalidades. Esperamos encontrar pelo menos:

- **Unitários:** validação de lead, algoritmo de distribuição, regras de alteração de status.
- **Integração:** POST /api/leads e POST /api/webhooks/callcenter.
- **Concorrência:** um teste que dispare múltiplas requisições simultâneas e verifique o resultado da distribuição.

Não é necessário possuir uma suíte gigantesca. Priorize as regras de negócio críticas.

24. Docker

A aplicação deverá possuir uma forma simples de ser executada, preferencialmente com **docker compose up**. O ambiente deverá disponibilizar pelo menos frontend, backend e banco de dados, além de qualquer outro serviço que sua arquitetura utilize.

25. Controle de versão

O repositório Git faz parte da avaliação.

- Faça commits incrementais ao longo do desenvolvimento, com mensagens que expliquem o que mudou. Não entregue o projeto em um único commit.
- Mantenha no repositório os artefatos de IA que você utilizou: arquivos de instrução ou regras (por exemplo CLAUDE.md, AGENTS.md), planos e especificações.
- Não reescreva o histórico antes de entregar.

26. README

O projeto deverá conter um README.md com:

- **Instalação e execução:** como rodar o projeto e os testes.
- **Arquitetura:** a arquitetura escolhida e o fluxo de um lead, do recebimento à atualização por webhook.
- **Decisões técnicas:** as principais decisões, incluindo as explicações pedidas nas seções anteriores.
- **Segurança:** quais medidas foram implementadas.
- **Escalabilidade:** o que você faria caso o volume aumentasse significativamente.
- **Limitações:** o que não foi implementado devido ao tempo disponível.
- **Melhorias futuras:** o que você faria em uma segunda versão.
- **Como usei IA:** quais ferramentas utilizou, o que delegou, o que a IA errou ou sugeriu e você rejeitou, e como validou o que foi gerado.

27. Priorização

Esta é uma parte importante do teste. Você não precisa implementar tudo com o mesmo nível de profundidade.

O núcleo do teste é: recebimento de leads, banco de dados, dispatcher com concorrência e webhook do call center. Essas partes devem estar funcionais e bem resolvidas. Para as demais, caso o tempo não seja suficiente:

- entregue uma implementação menor, porém funcional e bem estruturada;
- documente o que ficou pendente;
- explique como implementaria posteriormente.

Não adicione funcionalidades de forma superficial apenas para marcar todos os requisitos. A capacidade de identificar prioridades também faz parte da avaliação.

28. Stack

A escolha da stack é livre: utilize as tecnologias que domina, no backend, no frontend e no banco de dados (relacional). A escolha deve ser justificada, e não avaliamos positivamente o uso de uma tecnologia apenas por ser mais complexa.

29. O que será avaliado

Critério	Peso
Arquitetura	15
Banco de dados	15
Dispatcher / Concorrência	15
Backend / API	10
Segurança	10
Testes	10
Processo de desenvolvimento e uso de IA	10
Frontend	5
Qualidade do código	5
Infraestrutura	3
Documentação	2
Total	**100**

30. Apresentação técnica

Após a entrega, será realizada uma apresentação técnica de aproximadamente 45 minutos, com a aplicação em execução. Nela, você deverá:

- explicar a arquitetura, o banco de dados e o fluxo de um lead;
- demonstrar o funcionamento do dispatcher, dos webhooks, da autenticação e dos testes;
- responder a perguntas sobre suas decisões técnicas e sobre cenários que não foram implementados;
- realizar uma pequena alteração no código ao vivo, com as ferramentas que preferir, inclusive IA.

31. Entrega

Envie o link de um repositório Git contendo:

- código-fonte e histórico de commits;
- README.md;
- Docker / Docker Compose;
- testes.

O repositório deverá conter instruções suficientes para executar a aplicação.

32. Observação final

O objetivo deste teste não é criar a maior quantidade possível de funcionalidades. Queremos entender como você pensa, estrutura problemas, toma decisões e constrói software que possa evoluir.

Caso precise escolher entre implementar mais funcionalidades ou entregar uma solução menor, porém tecnicamente sólida, priorize a segunda opção e documente suas decisões.

Boa sorte!